Τελική Εξέταση - ΕΠΛ326
18/5/2021

Θέμα 1 (20 βαθμοί)

(1) Ποιος αλγόριθμος απαιτεί τους λιγότερους υπολιστικούς πόρους;			
(a) AES	(b) RSA	(c) DES	(d) RC4

(2) Ιδανικά ποιος είναι ο πιο ασφαλής τρόπος για να παράξουμε ένα κλειδί για συμμετρική κρυπτογράφηση των 128 bit:			
(a) να ρίξουμε ένα νόμισμα 128 φορές και να σημειώσουμε ‘1’ για γράμματα και ‘0’ για κορόνα	(b) να καλέσουμε τη rand() 128 φορές	(c) να καλέσουμε τη rand() μία φορά	(d) να διαλέξουμε δύο πρώτους αριθμούς και να τους πολλαπλασιάσουμε

(3) Η Αλίκη διαλέγει για ένα ζεύγος κλειδιών RSA να είναι e=3.			
(a) Δεν υπάρχει κανένα πρόβλημα, εφόσον ισχύει ότι gcd(e, φ(n))=1	(b) Θα πρέπει το e να είναι πολύ μεγαλύτερο	(c) Εξαρτάται από το d	(d) Εξαρτάται από το n

(4) Έχουμε μία κρυπτογραφική συνάρτηση κατακερματισμού (cryptographic hash function), για την οποία ισχύει H(m1) = H(m2). Ποιο από τα παρακάτω είναι το σωστό;			
(a) Αναγκαστικά θα πρέπει m1=m2	(b) Θα πρέπει να είναι εξαιρετικά δύσκολο να υπολογίσουμε το m1 και το m2	(c) Δεν μπορεί να συμβεί αυτό ποτέ	(d) Αν η H() είναι σωστή, τότε δεν θα πρέπει αυτό να συμβεί ποτέ

(5) Ποιο είναι το αποτέλεσμα της φ(46);

(6) Τα stack canaries προστατεύουν από			
(a) ROP	(b) γραμμικά overflows που αλλάζουν τη διεύθυνση επιστροφής στο stack	(c) οποιοδήποτε buffer overflow, το οποίο μπορεί να γίνει στο stack ή στο heap	(d) οποιοδήποτε buffer overflow ή use-after-free, το οποίο μπορεί να γίνει στο stack ή στο heap

(7) Οι επιθέσεις ROP γίνονται επειδή			
(a) είναι πιο γρήγορες	(b) η διεύθυνση του stack αλλάζει	(c) το stack και το heap είναι μη εκτελέσιμα και ο κώδικας δεν μπορεί να αλλάξει	(d) το shellcode δεν δουλεύει σε όλες τις πλατφόρμες

(8) Αντίθετα με τη strcpy(char *dst, char *src) η strncpy(char *dst, char *src, size_t len) δεν μπορεί να προκαλέσει buffer overflow. Συμφωνείτε;			
(a) Ναι, γιατί αντιγράφει μόνο όσα bytes χωρούν στο buffer υποδοχής (dst)	(b) Ναι, γιατί η υλοποιήση της strncpy() είναι ασφαλής	(c) Όχι, γιατί αν δεν υπολογιστεί σωστά το len, τότε μπορούμε και πάλι να έχουμε buffer overflow	(d) Εξαρτάται από τον compiler

(9) Το ASLR			
(a) σταματάει τα buffer overflows	(b) σταματάει τα code injections	(c) καταστρέφει τα gadgets	(d) αλλάζει τη θέση των gadgets

(10) Ένας dangling pointer μπορεί να βρίσκεται			
(a) μόνο στο stack	(b) μόνο στο heap	(c) είτε στο stack είτε στο heap	(d) αναγκαστικά και στο stack και στο heap

Θέμα 2 (20 βαθμοί)

Α) Η Αλίκη δεν έχει παρακολουθήσει το ΕΠΛ326 και υλοποίησε το RSA από μόνη της. Η υλοποίηση ήταν σωστή, άλλα στην χρήση της υλοποίησης έκανε κάποιες λανθασμένες επιλογές. Χρησιμοποίησε την υλοποίηση για να κρυπτογραφήσει ένα μήνυμα. Το αποτέλεσμα είναι το: **1595.**

Η Αλίκη, επειδή δεν έχει παρακολουθήσει το ΕΠΛ326, φρόντισε να κρύψει και τα δύο κλειδιά και να φανερώσει μόνο το **n = 4497.**

Βρείτε το μήνυμα που έχει κρυπτογραφήσει η Αλίκη, περιγράφοντας αναλυτικά τα βήματά σας (νούμερα και πράξεις χωρίς δικαιολόγηση δεν λαμβάνονται υπόψη). Θεωρήστε ότι το αρχικό μήνυμα είναι ένας **τριψήφιος ακέραιος.** Κατά την επίλυση του προβλήματος θα πρέπει να κάνετε κάποιες επιλογές, τις οποίες θα πρέπει να δικαιολογήσετε αναλυτικά. Τι δεν έχει κάνει η Αλίκη σωστά;

Β) Πόσες επαναλήψεις πρέπει να πραγματοποιήσει ο RC4 για να κρυπτογραφήσει το νούμερο 300; Εξηγήστε.

Θέμα 3 (20 βαθμοί)

Α) Θεωρήστε ένα σύστημα από το οποίο προμηθεύεστε λογισμικό. Επιλέγετε το πρόγραμμα που θέλετε να εγκαταστήσετε στον υπολογιστή σας και έπειτα το λογισμικό σας αποστέλλεται μέσω του δικτύου. Τι κακό μπορεί να γίνει κατά τη μεταφορά του αρχείου στον υπολογιστή σας; Περιγράψτε πώς θα κατασκευάσετε αυτό το σύστημα σωστά.

Β) Σχεδιάστε τον τρόπο λειτουργίας ενός αλγορίθμου κρυπτογράφησης ροής (stream cipher) και δείξτε πως ένας αλγόριθμος κρυπτογράφησης μπλοκ (block cipher) μπορεί να μετατραπεί σε stream cipher.

Θέμα 4 (20 βαθμοί)

Α) Η Αλίκη προσπαθεί να επιτεθεί με ROP σε ένα πρόγραμμα. Έχει βρεί την αδυναμία και έχει καταφέρει να βάλει στη μνήμη της υπό εκτέλεσης διεργασίας το ROP chain που έχουμε χρησιμοποιήσει στο μάθημα (επισυνάπτεται στο τέλος του δοκιμίου), ώστε να εκτελέσει το /bin/sh μέσω του execve(). Όμως, ενώ το ROP chain εκτελείται κανονικά, δεν εκτελεί το /bin/sh (η διεργασία τερματίζεται με crash). Παρακάτω μπορείτε να βρείτε μερικές εντολές που τρέχει η Αλίκη στο gdb. Μπορείτε να καταλάβετε γιατί το ROP chain δεν δουλεύει; Τι θα κάνατε για να το φτιάξετε;

```
(gdb) info file
Symbols from "/home/elathan/tmp/epl326/assignments/a5/filedb".
...
```

```
0x080eb060 - 0x080ec020 is .data
...
(gdb) x/4x 0x080eb060
0x80eb060: 0x90909090 0x90909090 0x90909d90 0x080bba88
```

B) Συμπληρώστε τα κενά. Αρχιτεκτονική: Intel 32-bit.

```
%ebp        %esp
0xffffd308, 0xffffd300 call   0x804846c <foo>
0xffffd308, 0xffffd2fc push   %ebp
          ,            mov    %esp,%ebp
          ,            sub    $0x18,%esp
          ,            movl   $0x8048330,-0xc(%ebp)
          ,            mov    -0xc(%ebp),%eax
          ,            call   *%eax
          ,            mov    %eax,%edx
          ,            mov    0x804a020,%eax
          ,            sub    $0x4,%esp
          , 0xffffd2dc push   %edx
          ,            push   $0x8048540
          ,            push   %eax
          ,            call   0x8048350 <fprintf@plt>
          ,            add    $0x10,%esp
          ,            nop
          , 0xffffd2e0 leave
0xffffd308, 0xffffd2fc ret
```

Θέμα 5 (20 βαθμοί)

Α) Σε ένα ROP chain μπορεί να βρεθεί το gadget `xchg %eax, %esp; ret`. Περιγράψτε αναλυτικά ποιος είναι ο ρόλος ενός τέτοιου gadget στο ROP chain.

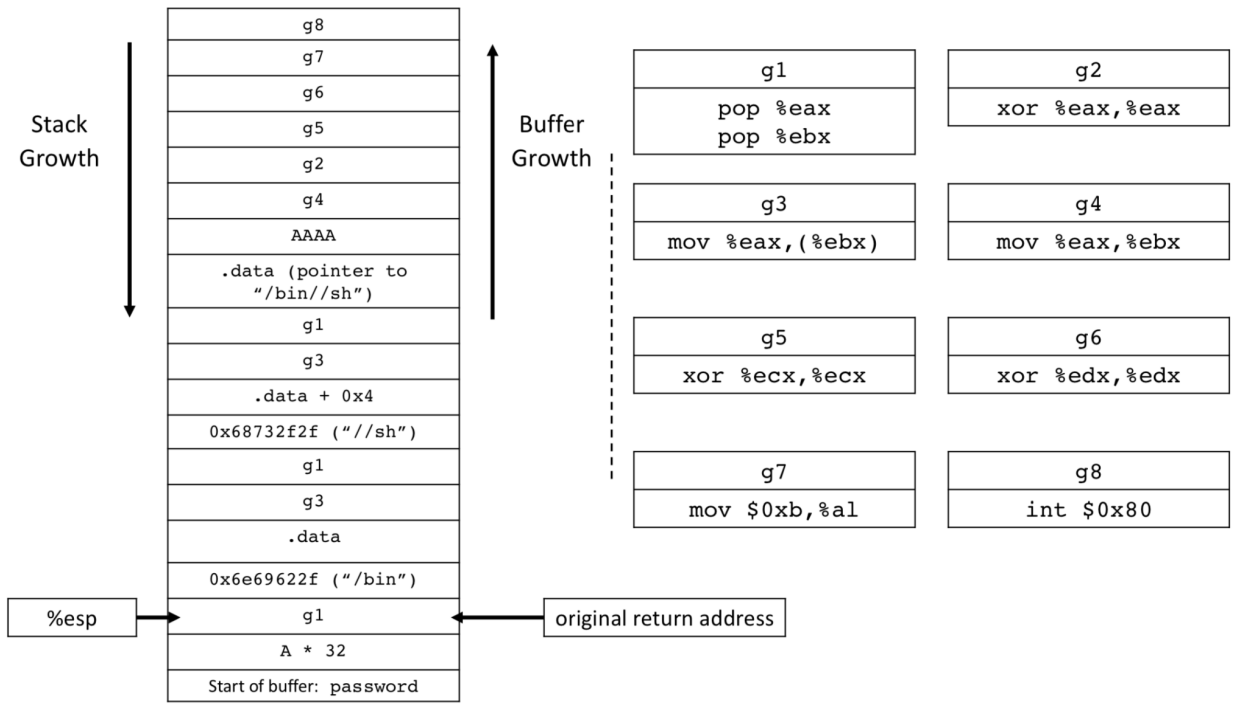
Β) Για το παρακάτω πρόγραμμα, βρείτε όλες τις ευπάθειες (vulnerabilities), και αναφέρετε σε ποιες γραμμές βρίσκονται. Περιγράψτε πώς μπορεί κάποιος να τις εκμεταλλευτεί και προτείνετε για κάθε ευπάθεια μια λύση. Δεν χρειάζεται να δώσετε για κάθε ευπάθεια συγκεκριμένη είσοδο, άλλα να περιγράψετε πώς μια είσοδος θα μπορούσε να κάνει επίθεση στο πρόγραμμα.

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. const signed char max_records = 100;
5.
6. struct record {
7.     int value;
8.     void (*display)(int v);
9. };
10.
11.struct record ** records;
12.
```

```
13.void render_record(int v) {
14.    fprintf(stderr, "Record number: %d\n", v);
15.}
16.
17.void init_records(void) {
18.
19.    records = malloc(sizeof(struct record)*max_records);
20.
21.    for (int i; i < max_records; i++) {
22.        struct record *r = malloc(sizeof *r);
23.        r->value = i;
24.        r->display = render_record;
25.        records[i] = r;
26.    }
27.
28.}
29.
30.void display_records(int i) {
31.
32.    fprintf(stderr, "\nDisplaying %d records.\n", i);
33.    for (int j; j < i; j++) {
34.        struct record *r = records[j];
35.        r->display(r->value);
36.        free(r);
37.    }
38.
39.}
40.
41.int main(int argc, char *argv[]) {
42.
43.    char name[32];
44.
45.    fprintf(stderr, "What is your name? ");
46.    scanf("%s", name);
47.    fprintf(stderr, "Welcome ");
48.    fprintf(stderr, name);
49.
50.    init_records();
51.
52.    unsigned char i = 0;
53.
54.    fprintf(stderr, ", how many records do you want to display? ");
55.
56.    scanf("%hhd", &i);
57.
58.    if (i > max_records) {
59.        display_records(max_records);
60.    }
```

```
    if ((char)(max_records - i) < 0) {
        /* We do not have that many records.  */
        fprintf(stderr, "Too many records (%hhd). Aborting.\n",
                (max_records-i));
        exit (1);
    }

    display_records(i);

    return 1;
}
```

Appendix - No more points below this line